CSE 482 Project

Predicting NASCAR Driver Performance Using Historical Race Statistics

Team Members: Arjun Ray, Aaryaman Bisht, Adhiraj Nimbalkar, Tolu Fashina, Noah Zemba

Collection and storing the data

Target Domain:

Our project focuses on NASCAR race analytics, specifically predicting race performance outcomes using historical driver and race data.

- Given a set of drivers participating in the same race, predict which driver will finish ahead.
- Given a set of drivers starting from different starting positions, will the starting affect race-end psoitions.
- Does recent form matter more than long term average?
- Does car driven affect driver position?

We used publicly available NASCAR data from SportsDataIO. The SportsDataIO NASCAR data dictionary was used to understand schema structure, variable meanings, and relationships between tables.

In [1]:
```
## Data Preprocessing
## Describe the original data quality after loading dataset -- details of th
## Clean and improve data, what strategies you’ve tried to improve it, and a
## Explain techniques used to clean and improve
```

In [2]:
```
import os
import json
import time
import requests
import pandas as pd
import math
from collections import defaultdict, deque
from itertools import combinations
import numpy as np
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_square
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

In [3]:
```
apikey="960411d9b7bf464794671691c80893fb"
if not apikey:
    raise ValueError("Missing API key. Please set SPORTSDATAIO_KEY in your e
BASE="https://api.sportsdata.io/nascar/v2/json"
HEADERS={"Ocp-Apim-Subscription-Key": apikey}
SEASONS=[2025]
SLEEP_SEC=0.25
os.makedirs("data/raw",exist_ok=True)
os.makedirs("data/processed",exist_ok=True)
```

In [4]:
```
df=pd.read_csv("sportsdataio-nascar-data-dictionary.csv")
df.head()
```

Out[4]:

	TableID	TableName	Name	DataType	Size	Nullable	Description	IsInCSV
0	8001000	Series	SeriesID	integer	32.0	False	The unique ID of this series	True
1	8001000	Series	Name	string	50.0	False	The name of this series	True
2	8002000	Race	RaceID	integer	32.0	False	The unique ID of this race	True
3	8002000	Race	SeriesID	integer	32.0	False	The unique ID of the series that this race is ...	True
4	8002000	Race	SeriesName	string	50.0	False	The name of the series that this race is assoc...	True

In [5]:
```
def get_json(url):
    r=requests.get(url, headers=HEADERS, timeout=60)
    if r.status_code!=200:
        raise RuntimeError(f"Request failed ({r.status_code}).\nURL: {url}\r
    return r.json()

def save_raw(obj, filename):
    path=os.path.join("data/raw", filename)
    with open(path,"w",encoding="utf-8") as f:
        json.dump(obj,f,indent=2)
    return path
```

In [6]:
```
all_races = []
for season in SEASONS:
    url=f"{BASE}/races/{season}"
    print("Fetching races for", season)
    races=get_json(url)
    save_raw(races, f"races_{season}.json")
    all_races+=races
    time.sleep(SLEEP_SEC)

races_df=pd.DataFrame(all_races)
print("Raw races rows:", len(races_df), "cols:", len(races_df.columns))

wc=["RaceID", "SeriesID", "SeriesName", "Season", "Name", "Day", "DateTime",
keep_cols=[c for c in wc if c in races_df.columns]
races_df=races_df[keep_cols].copy()
if "DateTime" in races_df.columns:
    races_df["DateTime"] = pd.to_datetime(races_df["DateTime"], errors="coer

if "IsOver" in races_df.columns:
    races_df=races_df[races_df["IsOver"]==True].copy()
races_df=races_df.dropna(subset=["RaceID"]).drop_duplicates(subset=["RaceID"
out_path = "data/processed/races.csv"
races_df.to_csv(out_path, index=False)
print("Saved:",out_path,"rows=",len(races_df))
```

```
Fetching races for 2025
Raw races rows: 99 cols: 20
Saved: data/processed/races.csv rows= 99
```

In [7]:
```
drivers_url = f"{BASE}/Drivers"
print("Fetching drivers...")
drivers = get_json(drivers_url)
save_raw(drivers, "drivers.json")
drivers_df = pd.DataFrame(drivers)
wc=["DriverID", "FirstName", "LastName", "Number", "Team", "Manufacturer"]
keep=[c for c in wc if c in drivers_df.columns]
drivers_df = drivers_df[keep].drop_duplicates(subset=["DriverID"])
out_path = "data/processed/drivers.csv"
drivers_df.to_csv(out_path, index=False)
print("Saved:", out_path, "rows=", len(drivers_df))
```

```
Fetching drivers...
Saved: data/processed/drivers.csv rows= 848
```

In [8]:
```
races_final = pd.read_csv("data/processed/races.csv")
race_ids = races_final["RaceID"].dropna().astype(int).tolist()
all_rows = []
failed = []
for rid in race_ids:
    url=f"{BASE}/raceresultfinal/{rid}"
    try:
        rr = get_json(url)
        save_raw(rr, f"raceresultfinal_{rid}.json")
        race_obj = rr.get("Race", {}) if isinstance(rr, dict) else {}
        driver_races = rr.get("DriverRaces", []) if isinstance(rr, dict) els
        for dr in driver_races:
```

```
            all_rows.append({
                "RaceID": race_obj.get("RaceID", rid),
                "Season": race_obj.get("Season"),
                "SeriesName": race_obj.get("SeriesName"),
                "DateTime": race_obj.get("DateTime"),
                "Track": race_obj.get("Track"),
                "DriverID": dr.get("DriverID"),
                "StartPosition": dr.get("StartPosition") or dr.get("Starting
                "FinishPosition": dr.get("FinishPosition") or dr.get("FinalF
                "Laps": dr.get("Laps") or dr.get("LapsCompleted"),
                "Points": dr.get("Points"),
            })

        time.sleep(SLEEP_SEC)

    except Exception as e:
        failed.append((rid, str(e)))

results_df = pd.DataFrame(all_rows)
print("Raw results rows:", len(results_df))
results_df["DateTime"] = pd.to_datetime(results_df["DateTime"], errors="coer
num_cols = ["RaceID", "DriverID", "StartPosition", "FinishPosition", "Laps",
for col in num_cols:
    if col in results_df.columns:
        results_df[col] = pd.to_numeric(results_df[col], errors="coerce")
results_df = results_df.dropna(subset=["RaceID", "DriverID", "FinishPositior
results_df = results_df.drop_duplicates(subset=["RaceID", "DriverID"])
results_df["Top10"] = (results_df["FinishPosition"] <= 10).astype(int)

out_path = "data/processed/race_results.csv"
results_df.to_csv(out_path, index=False)

print("Saved:", out_path, "rows=", len(results_df))
print("Failed races:", len(failed))
if failed:
    print("First failure:", failed[0])
```

```
Raw results rows: 3630
Saved: data/processed/race_results.csv rows= 3584
Failed races: 0
```

We retrieved the data using the API and stored it in pandas DataFrames. From the retrieved data, a clean and structured data set was formed by combining the necessary data. This was done by matching the race columns and ensuring the data sets were compatible.

The original data set had missing information and a few columns that were not necessary for the model's predictions. The unwanted attributes were removed from the data set. The columns were also made uniform in name and data type for a smooth transition in the next phase.

New columns were added to the data set for easier analysis. For instance, new columns were added for easier classification. The data was also reorganized so that each row corresponds to a driver and a race.

This resulted in a clean and tabular data set ready for machine learning. The clean data set was ready for the next phase in the process.

In [9]:
```
results = pd.read_csv("data/processed/race_results.csv")
drivers = pd.read_csv("data/processed/drivers.csv")
races = pd.read_csv("data/processed/races.csv")

results["DateTime"] = pd.to_datetime(results["DateTime"], errors="coerce")
races["DateTime"] = pd.to_datetime(races["DateTime"], errors="coerce")

for c in ["RaceID", "DriverID", "StartPosition", "FinishPosition", "Laps", "
    if c in results.columns:
        results[c] = pd.to_numeric(results[c], errors="coerce")

drivers["DriverName"] = (drivers["FirstName"].fillna("") + " " + drivers["La
race_info = races[["RaceID", "Name", "Track", "DateTime", "Season"]].rename(
driver_info = drivers[["DriverID", "DriverName", "Team", "Manufacturer"]]

df = results.merge(race_info, on="RaceID", how="left", suffixes=("", "_race"
df = df.merge(driver_info, on="DriverID", how="left")
df["Track"] = df["Track_race"].combine_first(df["Track"])
df["DateTime"] = df["DateTime_race"].combine_first(df["DateTime"])
df["Season"] = df["Season_race"].combine_first(df["Season"])
df = df.drop(columns=[c for c in ["Track_race", "DateTime_race", "Season_rac
df = df.dropna(subset=["RaceID", "DriverID", "StartPosition", "FinishPositic
df = df.drop_duplicates(["RaceID", "DriverID"]).sort_values(["DateTime", "Ra

df.head()
```

Out[9]:

	RaceID	Season	SeriesName	DateTime	Track	DriverID	StartPosition	FinishP
0	1002	2025	NASCAR Cup	2025-02-02 20:00:00	Bowman Gray Stadium	80000261	0.8	
1	1002	2025	NASCAR Cup	2025-02-02 20:00:00	Bowman Gray Stadium	80000263	3.3	
2	1002	2025	NASCAR Cup	2025-02-02 20:00:00	Bowman Gray Stadium	80000268	2.1	
3	1002	2025	NASCAR Cup	2025-02-02 20:00:00	Bowman Gray Stadium	80000274	0.5	
4	1002	2025	NASCAR Cup	2025-02-02 20:00:00	Bowman Gray Stadium	80000284	1.1	

Feature Selection

In [10]:
```
def make_features(df):
    hist = defaultdict(list)
    track_hist = defaultdict(list)
    recent = defaultdict(lambda: deque(maxlen=5))
    rows = []

    for _, r in df.sort_values(["DateTime", "RaceID", "DriverID"]).iterrows(
        d = int(r["DriverID"])
        t = r["Track"] if pd.notna(r["Track"]) else "Unknown"
        prev = hist[d]
        prev_track = track_hist[(d, t)]

        rows.append({
            **r.to_dict(),
            "prior_races": len(prev),
            "prior_avg_finish": np.mean([x["finish"] for x in prev]) if prev
            "prior_avg_start": np.mean([x["start"] for x in prev]) if prev e
            "prior_top10_rate": np.mean([x["finish"] <= 10 for x in prev]) i
            "prior_avg_points": np.mean([x["points"] for x in prev if pd.not
            "recent5_avg_finish": np.mean(recent[d]) if len(recent[d]) else
            "track_prior_races": len(prev_track),
            "track_avg_finish": np.mean(prev_track) if prev_track else np.na
        })
        prev.append({
            "finish": float(r["FinishPosition"]),
            "start": float(r["StartPosition"]),
            "points": float(r["Points"]) if pd.notna(r["Points"]) else np.na
        })
        track_hist[(d, t)].append(float(r["FinishPosition"]))
        recent[d].append(float(r["FinishPosition"]))

    return pd.DataFrame(rows)

feat = make_features(df)
feat = feat[feat["prior_races"] >= 5].copy()

race_order = feat[["RaceID", "DateTime"]].drop_duplicates().sort_values(["Da
split = int(len(race_order) * 0.8)
train_ids = set(race_order.iloc[:split]["RaceID"])
test_ids = set(race_order.iloc[split:]["RaceID"])

train = feat[feat["RaceID"].isin(train_ids)].copy()
test = feat[feat["RaceID"].isin(test_ids)].copy()
y_train = train["FinishPosition"]
y_test = test["FinishPosition"]
cols = ["StartPosition", "prior_races", "prior_avg_finish", "prior_avg_start
lin_full = Pipeline([("imp", SimpleImputer(strategy="median")),("model", Lir
tree_full = Pipeline([("imp", SimpleImputer(strategy="median")),("model", De
lin_full.fit(train[cols], y_train)
tree_full.fit(train[cols], y_train)

pred_lin = lin_full.predict(test[cols])
```

```
pred_tree = tree_full.predict(test[cols])
print("train rows:", len(train), "test rows:", len(test))
feat[cols + ["FinishPosition"]].head()
```

```
train rows: 2175 test rows: 631
```

Out[10]:

	StartPosition	prior_races	prior_avg_finish	prior_avg_start	prior_top10_rate	pr
333	1.1	5	8.42	6.68	0.8	
341	2.2	5	8.90	13.74	0.4	
347	1.2	5	11.68	4.08	0.6	
353	2.0	5	1.82	2.88	1.0	
363	0.5	5	16.90	14.40	0.6	

Given a set of drivers participating in the same race, predict which driver will finish ahead

In [11]:

```
def make_pairs(x):
    rows = []
    for race_id, g in x.groupby("RaceID"):
        recs = g[["RaceID", "FinishPosition"] + cols].to_dict("records")
        for a, b in combinations(recs, 2):
            row = {"RaceID": race_id, "label": int(a["FinishPosition"] < b["
            for c in cols:
                row[c + "_diff"] = a[c] - b[c] if pd.notna(a[c]) and pd.notr
            rows.append(row)
    return pd.DataFrame(rows)

pairs = make_pairs(feat)
train_pairs = pairs[pairs["RaceID"].isin(train_ids)].copy()
test_pairs = pairs[pairs["RaceID"].isin(test_ids)].copy()
pair_cols = [c + "_diff" for c in cols]

logit = Pipeline([("imp", SimpleImputer(strategy="median")),("scale", Standa
ctree = Pipeline([("imp", SimpleImputer(strategy="median")),("model", Decisi
logit.fit(train_pairs[pair_cols], train_pairs["label"])
ctree.fit(train_pairs[pair_cols], train_pairs["label"])

pred_logit = logit.predict(test_pairs[pair_cols])
prob_logit = logit.predict_proba(test_pairs[pair_cols])[:, 1]
pred_ctree = ctree.predict(test_pairs[pair_cols])
prob_ctree = ctree.predict_proba(test_pairs[pair_cols])[:, 1]

pair_results = pd.DataFrame([["LogisticRegression", accuracy_score(test_pair

print("Results for finishing ahead")
display(pair_results.round(4))
```

```
Results for finishing ahead
```

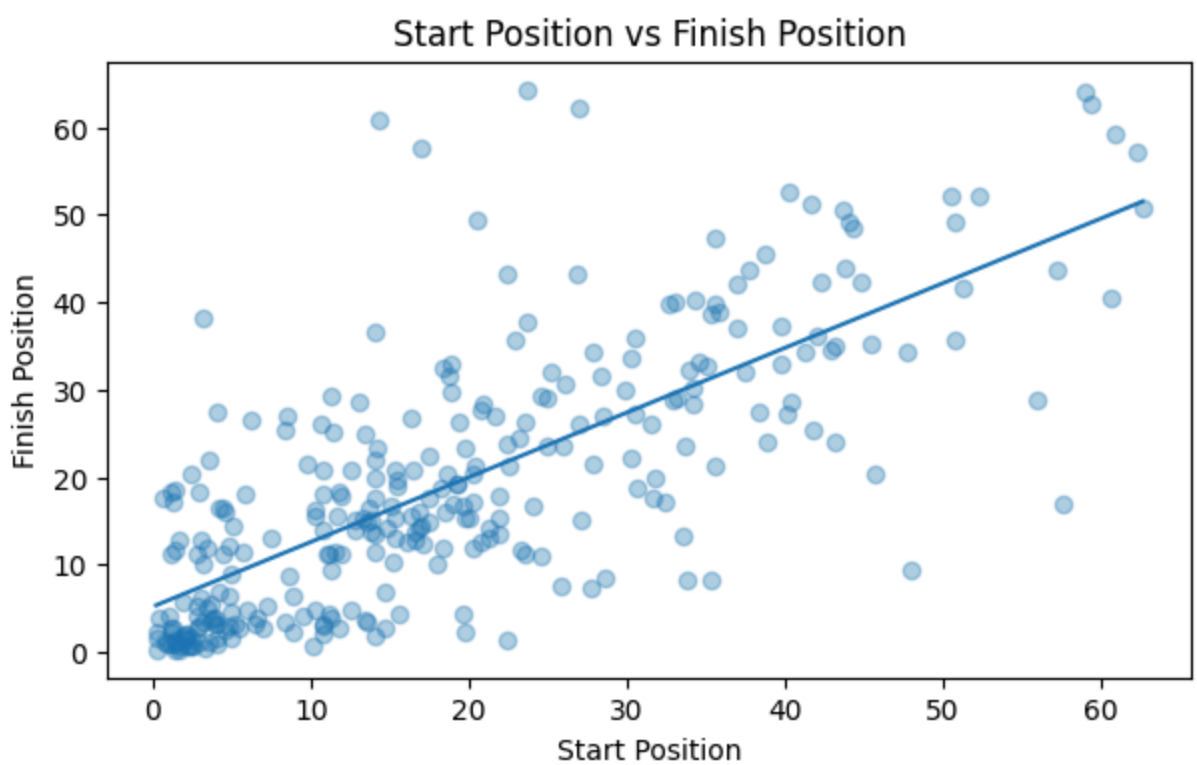
	model	accuracy	roc_auc
0	LogisticRegression	0.6878	0.7569
1	DecisionTree	0.6682	0.7241

Given a set of drivers starting from different starting positions, will the starting affect race-end psoitions.

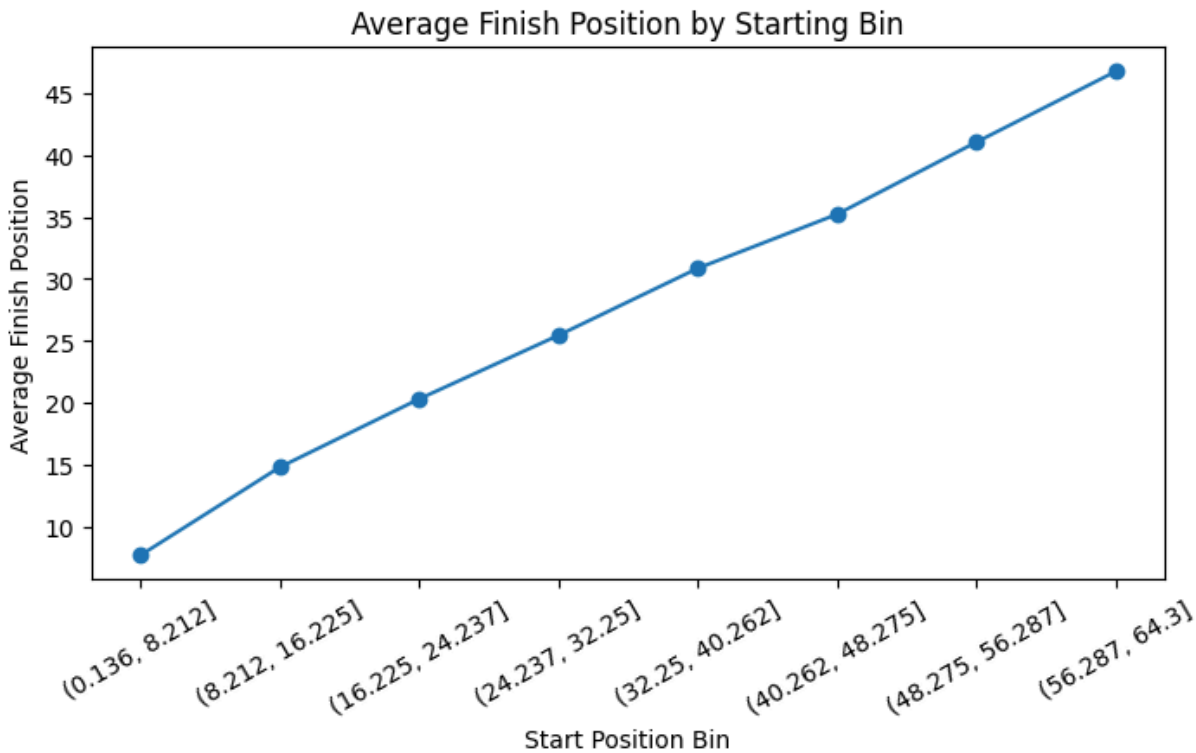
In [12]:
```
spearman_all = feat["StartPosition"].corr(feat["FinishPosition"], method="sp
spearman_test = test["StartPosition"].corr(test["FinishPosition"], method="s
start_train = train[["StartPosition"]]
start_test = test[["StartPosition"]]
lin_start = Pipeline([("imp", SimpleImputer(strategy="median")),("model", Li

tree_start = Pipeline([("imp", SimpleImputer(strategy="median")),("model", D

lin_start.fit(start_train, y_train)
tree_start.fit(start_train, y_train)

pred_lin_start = lin_start.predict(start_test)
pred_tree_start = tree_start.predict(start_test)

no_start_cols = [c for c in cols if c != "StartPosition"]

lin_no_start = Pipeline([("imp", SimpleImputer(strategy="median")),("model",

tree_no_start = Pipeline([("imp", SimpleImputer(strategy="median")),("model"

lin_no_start.fit(train[no_start_cols], y_train)
tree_no_start.fit(train[no_start_cols], y_train)

pred_no_start_lin = lin_no_start.predict(test[no_start_cols])
pred_no_start_tree = tree_no_start.predict(test[no_start_cols])

start_results = pd.DataFrame([
    ["Spearman corr (all)", spearman_all],
    ["Spearman corr (test)", spearman_test],
    ["Linear start-only MAE", mean_absolute_error(y_test, pred_lin_start)],
    ["Tree start-only MAE", mean_absolute_error(y_test, pred_tree_start)],
    ["Linear full model MAE", mean_absolute_error(y_test, pred_lin)],
    ["Linear without StartPosition MAE", mean_absolute_error(y_test, pred_no
    ["Tree full model MAE", mean_absolute_error(y_test, pred_tree)],
    ["Tree without StartPosition MAE", mean_absolute_error(y_test, pred_no_s
], columns=["metric", "value"])

display(start_results.round(4))
```

	metric	value
0	Spearman corr (all)	0.6588
1	Spearman corr (test)	0.6926
2	Linear start-only MAE	7.9445
3	Tree start-only MAE	8.1298
4	Linear full model MAE	7.9923
5	Linear without StartPosition MAE	10.6326
6	Tree full model MAE	9.9613
7	Tree without StartPosition MAE	11.8526

In [13]:
```
sample = test.sample(min(len(test), 300), random_state=42)

plt.figure(figsize=(7, 4))
plt.scatter(sample["StartPosition"], sample["FinishPosition"], alpha=0.35)

m, b = np.polyfit(sample["StartPosition"], sample["FinishPosition"], 1)
x = np.linspace(sample["StartPosition"].min(), sample["StartPosition"].max()
plt.plot(x, m * x + b)

plt.xlabel("Start Position")
plt.ylabel("Finish Position")
plt.title("Start Position vs Finish Position")
plt.show()
```



In [14]:
```
start_bins = test.copy()
start_bins["start_bin"] = pd.cut(start_bins["StartPosition"], bins=8)

bin_means = start_bins.groupby("start_bin", observed=False)["FinishPosition"

plt.figure(figsize=(8, 4))
plt.plot(range(len(bin_means)), bin_means.values, marker="o")
plt.xticks(range(len(bin_means)), [str(x) for x in bin_means.index], rotatic
plt.xlabel("Start Position Bin")
plt.ylabel("Average Finish Position")
plt.title("Average Finish Position by Starting Bin")
plt.show()
```



We compared several models to predict race outcomes. For finishing positions, Linear Regression performed best, giving the lowest error on test races. For predicting which driver finishes ahead, Logistic Regression worked best, with about 68% accuracy and a ROC-AUC of around 0.75. Overall, the models were able to pick up useful patterns from past race data and make reasonably accurate predictions.

Does recent form matter more than long term average?

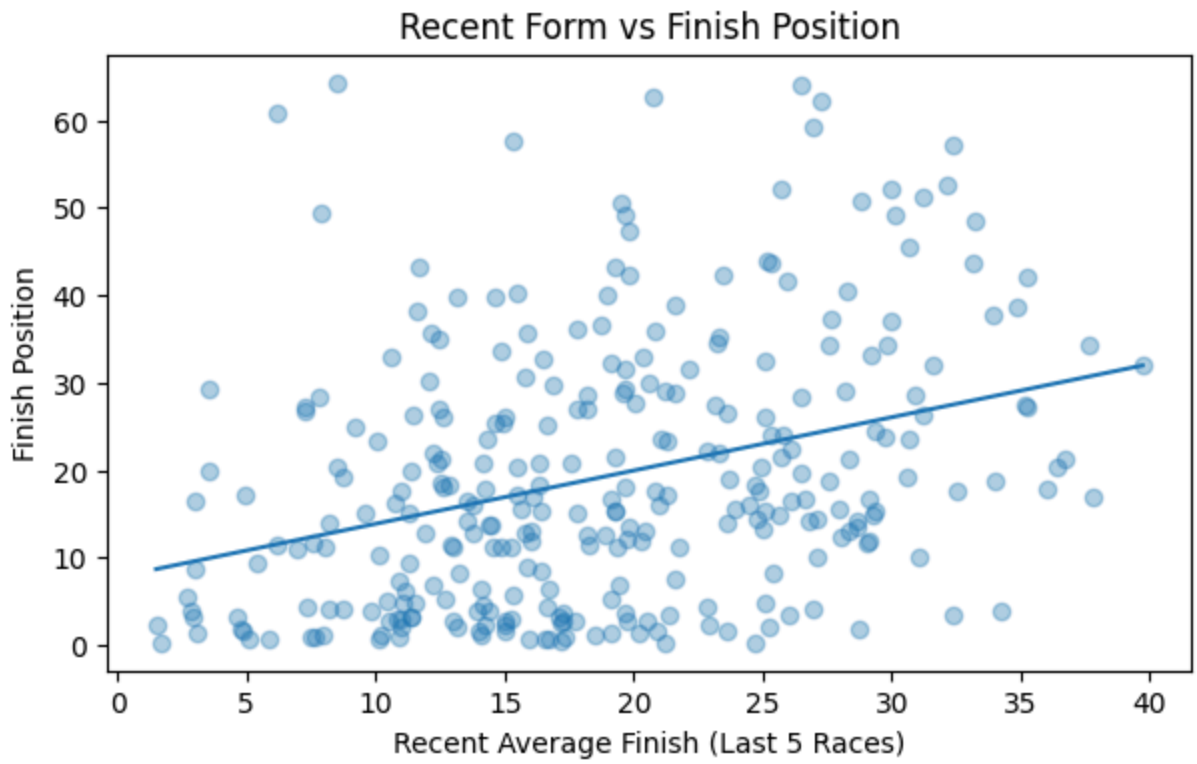
In [15]:
```
recent_cols = ["StartPosition", "prior_races", "recent5_avg_finish"]
career_cols = ["StartPosition", "prior_races", "prior_avg_finish"]
both_cols = ["StartPosition", "prior_races", "recent5_avg_finish", "prior_av
feature_sets = {
    "recent_only": recent_cols,
    "career_only": career_cols,
    "recent_plus_career": both_cols
}
```

```
recent_results = []
for name, use_cols in feature_sets.items():
    X_train = train[use_cols]
    X_test = test[use_cols]
    y_train = train["FinishPosition"]
    y_test = test["FinishPosition"]
    lin_model = Pipeline([("imp", SimpleImputer(strategy="median")),("model"

    tree_model = Pipeline([("imp", SimpleImputer(strategy="median")),("model
    lin_model.fit(X_train, y_train)
    tree_model.fit(X_train, y_train)
    pred_lin = lin_model.predict(X_test)
    pred_tree = tree_model.predict(X_test)
    recent_results.append([name,"LinearRegression",mean_absolute_error(y_tes

    recent_results.append([name,"DecisionTree",mean_absolute_error(y_test, p
recent_results = pd.DataFrame(recent_results,columns=["feature_set", "model"
display(recent_results.round(4))
```

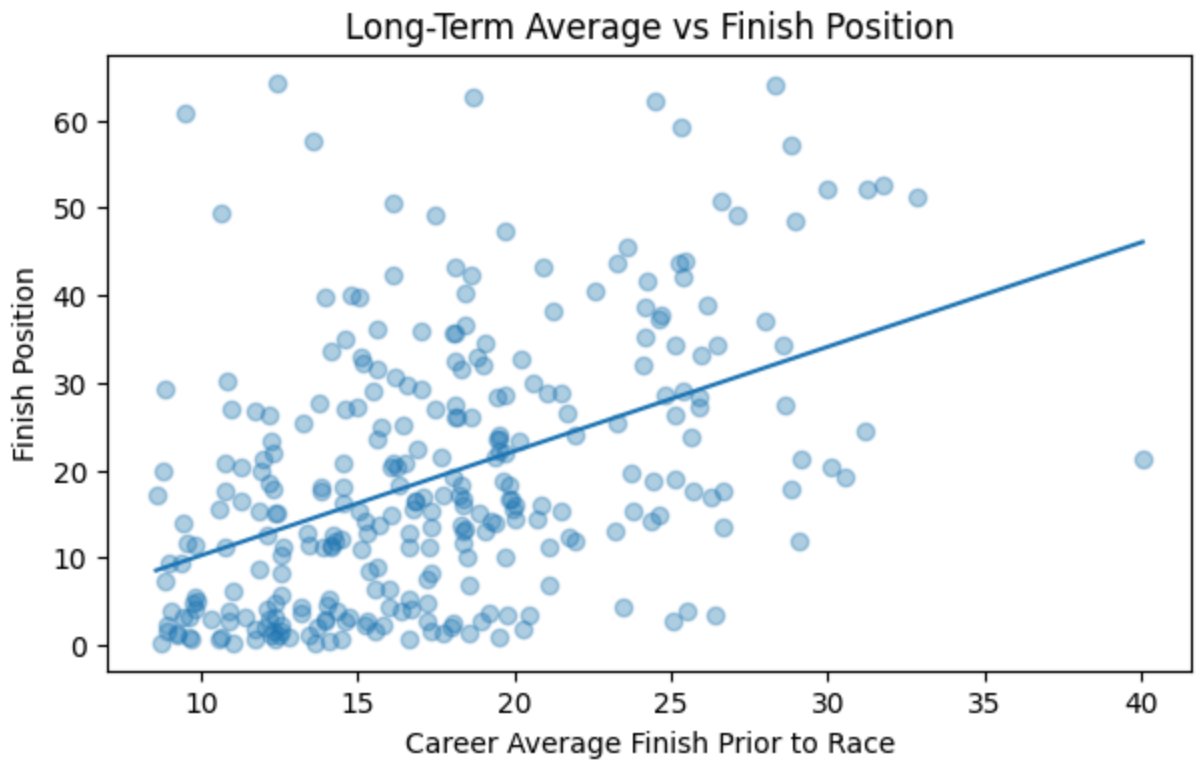
	feature_set	model	mae	rmse
0	recent_only	LinearRegression	7.9850	10.3544
2	career_only	LinearRegression	7.9879	10.3390
4	recent_plus_career	LinearRegression	8.0085	10.3032
3	career_only	DecisionTree	8.6778	11.3668
5	recent_plus_career	DecisionTree	9.7500	12.7153
1	recent_only	DecisionTree	9.8820	12.9179

In [16]:

```
sample_recent = test.dropna(subset=["recent5_avg_finish", "FinishPosition"])
    min(300, test.dropna(subset=["recent5_avg_finish", "FinishPosition"]).sh
    random_state=42)

plt.figure(figsize=(7, 4))
plt.scatter(sample_recent["recent5_avg_finish"], sample_recent["FinishPositi

m, b = np.polyfit(sample_recent["recent5_avg_finish"], sample_recent["Finish
x = np.linspace(sample_recent["recent5_avg_finish"].min(), sample_recent["re
plt.plot(x, m * x + b)

plt.xlabel("Recent Average Finish (Last 5 Races)")
plt.ylabel("Finish Position")
plt.title("Recent Form vs Finish Position")
plt.show()
```



In [17]:
```
sample_career = test.dropna(subset=["prior_avg_finish", "FinishPosition"]).s
    min(300, test.dropna(subset=["prior_avg_finish", "FinishPosition"]).shap
    random_state=42)

plt.figure(figsize=(7, 4))
plt.scatter(sample_career["prior_avg_finish"], sample_career["FinishPositior

m, b = np.polyfit(sample_career["prior_avg_finish"], sample_career["FinishPo
x = np.linspace(sample_career["prior_avg_finish"].min(), sample_career["pric
plt.plot(x, m * x + b)

plt.xlabel("Career Average Finish Prior to Race")
plt.ylabel("Finish Position")
plt.title("Long-Term Average vs Finish Position")
plt.show()
```

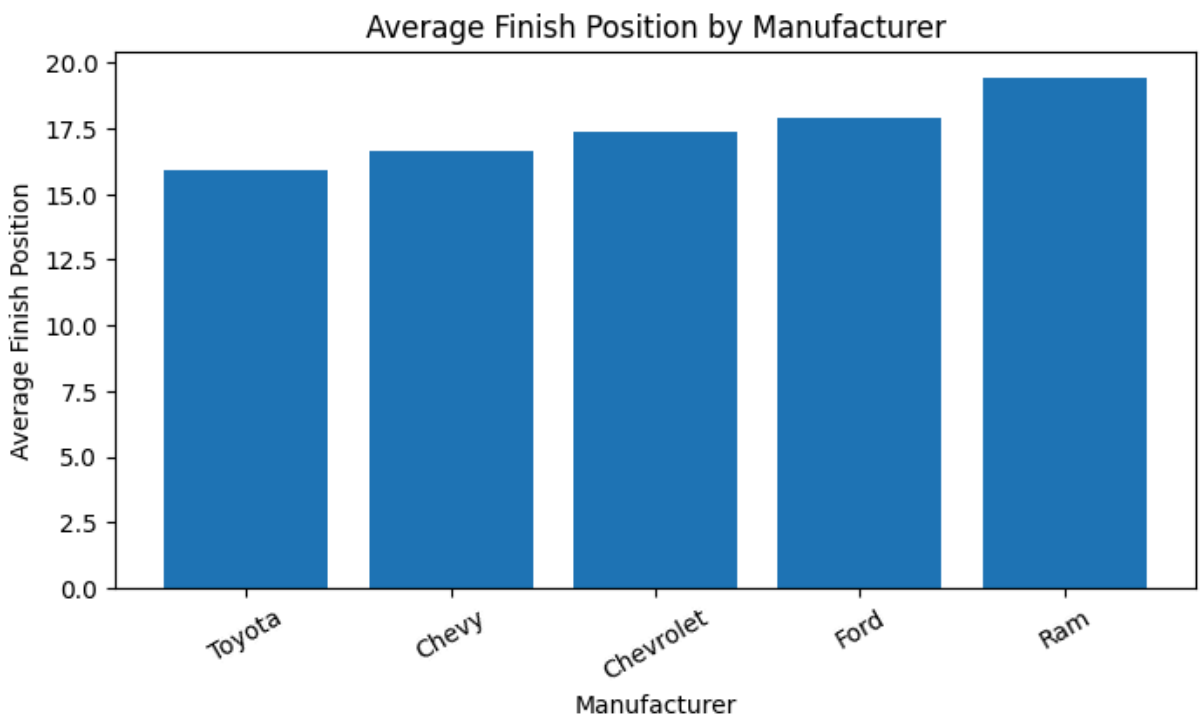


Does car driven affect driver position?

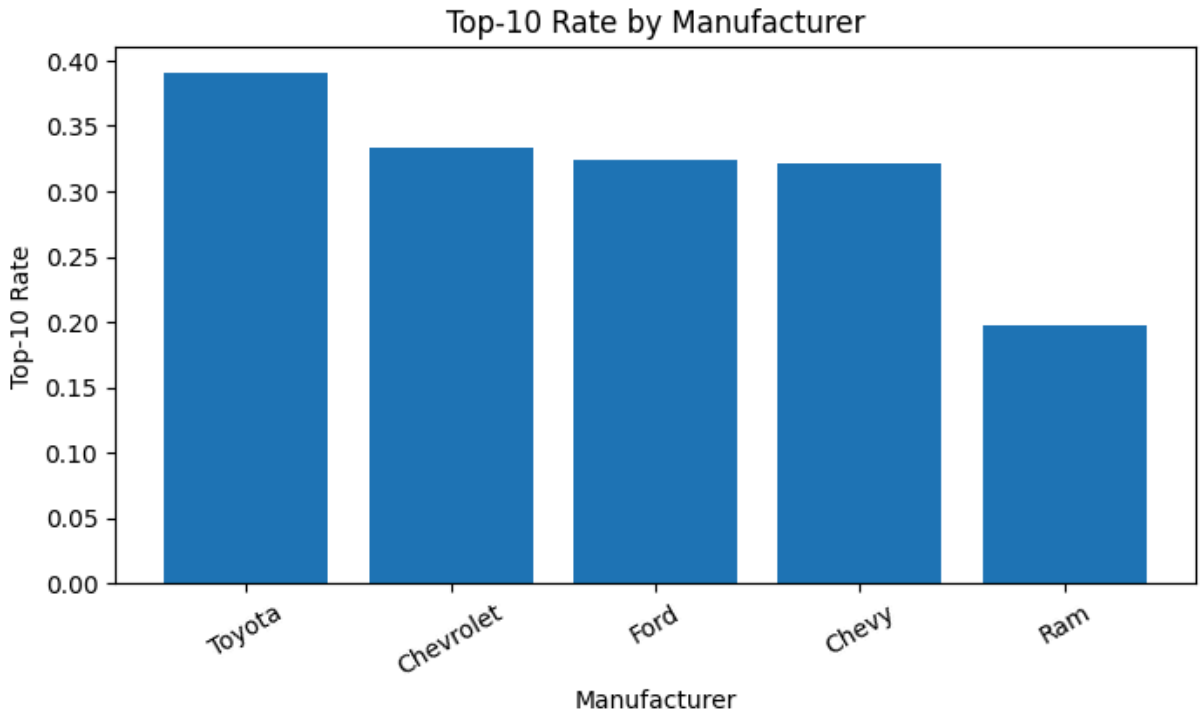
In [18]:
```
base_cols = ["StartPosition", "prior_races", "prior_avg_finish", "recent5_av
manu_cols = base_cols + ["Manufacturer"]
car_results = []
for name, use_cols in {"without_manufacturer": base_cols, "with_manufacturer
    numeric_cols = [c for c in use_cols if c != "Manufacturer"]
    categorical_cols = [c for c in use_cols if c == "Manufacturer"]

    preprocessor = ColumnTransformer([("num", Pipeline([("imp", SimpleImpute
    lin_model = Pipeline([("prep", preprocessor),("model", LinearRegression(

    tree_model = Pipeline([("prep", preprocessor),("model", DecisionTreeRegr
    X_train = train[use_cols]
    X_test = test[use_cols]
    y_train = train["FinishPosition"]
    y_test = test["FinishPosition"]

    lin_model.fit(X_train, y_train)
    tree_model.fit(X_train, y_train)

    pred_lin = lin_model.predict(X_test)
    pred_tree = tree_model.predict(X_test)

    car_results.append([name,"LinearRegression",mean_absolute_error(y_test,
    car_results.append([name,"DecisionTree",mean_absolute_error(y_test, prec
car_results = pd.DataFrame(car_results,columns=["feature_set", "model", "mae
display(car_results.round(4))
```

	feature_set	model	mae	rmse
2	with_manufacturer	LinearRegression	8.0083	10.3320
0	without_manufacturer	LinearRegression	8.0085	10.3032
3	with_manufacturer	DecisionTree	9.7132	12.7055
1	without_manufacturer	DecisionTree	9.7500	12.7153

In [19]:
```
manu_plot = feat.dropna(subset=["Manufacturer", "FinishPosition"]).copy()

top_manus = manu_plot["Manufacturer"].value_counts().head(8).index
manu_plot = manu_plot[manu_plot["Manufacturer"].isin(top_manus)]

manu_means = manu_plot.groupby("Manufacturer")["FinishPosition"].mean().sort

plt.figure(figsize=(8, 4))
plt.bar(manu_means.index, manu_means.values)
plt.xlabel("Manufacturer")
plt.ylabel("Average Finish Position")
plt.title("Average Finish Position by Manufacturer")
plt.xticks(rotation=30)
plt.show()
```



In [20]:
```
manu_top10 = manu_plot.groupby("Manufacturer")["FinishPosition"].apply(lambc
plt.figure(figsize=(8, 4))
plt.bar(manu_top10.index, manu_top10.values)
plt.xlabel("Manufacturer")
plt.ylabel("Top-10 Rate")
plt.title("Top-10 Rate by Manufacturer")
plt.xticks(rotation=30)
plt.show()
```



We also looked at whether recent form and car type affect performance. For recent vs long-term form, Linear Regression showed that recent performance slightly outperformed long-term averages, but the difference was small, and combining both did not improve results much. This suggests recent form matters a bit more, but both contain similar information. For car manufacturer, adding it to the model did not improve performance, as errors stayed almost the same with and without it. Overall, recent form has a small impact, while car type does not seem to significantly affect finishing position.